EJERCICIO 1

Utilizando una pila, comprobar si una expresión que contiene paréntesis está balanceada o no.

```
unit uPilaChar;

interface
   type
     tnodo = record
       info: char;
       ant: ^tnodo;
     end;
     tPilaChar = ^tnodo;

   procedure initialize(var p:tPilaChar);
   procedure push(var p:tPilaChar; x:char);
   function pop(var p:tPilaChar): char;
   function peek(p:tPilaChar): char;
   function is_empty(p:tPilaChar): boolean;

implementation

   procedure initialize(var p:tPilaChar);
   begin
     p := nil;
   end;

   ... etc.

(el resto de la implementación de la unit es previsible a la vista de la
implementación ya incluida en los apuntes para los integer)

program ejercicio_propuestos_pilas1;
  { Resuelto en el programa principal para centrarnos en el algoritmo y
  el uso de la pila. La mejor implementación sería una función booleana }

uses uPilaChar, crt;

const { Datos de prueba, usar uno u otro para probar el código }
  {EXPRESION = '3 + (8 * 9 + (34 – 2) – 72/ (3 – 56))';}
  EXPRESION = '3 + 8 * 9 + (34 – 2) – 72/ (3 – 56))';

var
   p: tPilaChar;
   i: integer;
   balanceada: boolean;

begin
  i:= 1;
  initialize(p);
  balanceada:= true;
  while (i<=length(EXPRESION)) and (balanceada) do
  begin
    case EXPRESION[i] of
     '(': push(p,'(');
     ')': if is_empty(p) then balanceada:=false else pop(p);
    end; {en otro caso, no hacer nada}
    i:= i+1;
  end;
  if is_empty(p) and balanceada then writeln('OK expresion balanceada')
  else writeln('NO OK - Expresion erronea');
  repeat until keypressed
end.
```

EJERCICIO 1bis

Utilizando una pila, comprobar si una expresión que contiene **paréntesis y corchetes** está balanceada o no.

```
program ejercicio_propuestos_pilas1bis;

uses uPilaChar, crt;
{ Usamos la misma unit uPilaChar del ej. 1, por lo que no se vuelve a incluir
  su implementación }

function concuerdan(c1,c2: char): boolean;
begin
  concuerdan := ((c1='[') and (c2=']')) or ((c1='(') and (c2=')'))
end;

function expresion_balanceada(expresion: string): boolean;
var
   p: tPilaChar;
   i: integer;
   balanceada: boolean;
begin
  i:= 1;
  initialize(p);
  balanceada:= true;
  while (i<=length(expresion)) and (balanceada) do
  begin
    case EXPRESION[i] of
     '(','[': push(p,expresion[i]);
     ')',']': if is_empty(p) then balanceada:=false
          else
              if concuerdan(peek(p),expresion[i]) then pop(p)
              else balanceada:=false;
    end; {en otro caso, no hacer nada}
    i:= i+1;
  end;
  expresion_balanceada := is_empty(p) and balanceada;
end;

var
   expresion_prueba: string[255];

   { Datos para pruebas (cortar pegar y probar):
    OK ->    '3 + [(8 * 9 + (34 - 2) - 72/ (3 - 56))]' // '[()]()' // '[[()]]'
    NO OK -> '[3 + 8 * 9 + (34] - 2) - 72/ (3 - 56))' // '[()])' // '[()]]'
   }

begin
   expresion_prueba := '[3 + 8 * 9 + (34] - 2) - 72/ (3 - 56))';
   if expresion_balanceada(expresion_prueba) then
      writeln('Expresion OK - Esta balanceada')
   else
       writeln('Expresion erronea - NO esta balanceada');
   repeat until keypressed
end.
```

EJERCICIO 2

Implementar una pila de libros, guardando de cada libro su título, autor, ISBN, idioma y número de páginas. Una vez implementado lo anterior, implementa un subprograma que recoloque los libros de la pila al revés de como están actualmente.

```
unit uPilaGenerica; { Almacenada en archivo uPilaGenerica.pas }

interface

   uses uTElemento;

   type
     tnodo = record
       info: TElemento;
       ant: ^tnodo;
     end;
     tPila = ^tnodo;

   procedure initialize(var p:tPila);
   procedure push(var p:tPila; x:TElemento);
   procedure pop(var p:tPila);
   function peek(p:tPila): TElemento;
   function is_empty(p:tPila): boolean;
   procedure clear(var p:tPila);

implementation

   procedure initialize(var p:tPila);
   begin
     p := nil;
   end;

   function is_empty(p:tPila): boolean;
   begin
     is_empty := p = nil;
   end;

   procedure push(var p:tPila; x:TElemento);
   var
     new_node: ^tnodo;
   begin
     new(new_node);
     assign(new_node^.info,x);
     new_node^.ant := p;
     p:= new_node;
   end;

   procedure pop(var p:tPila);
   var
     aux: ^tnodo;
   begin
     if not is_empty(p) then begin
       aux := p;
       p := p^.ant;
       dispose(aux);
     end;
   end;

   function peek(p:tPila): TElemento;
   begin
     if not is_empty(p) then
        assign(peek,p^.info);
   end;
```

```
   procedure clear(var p:tPila);
   begin
     while not is_empty(p) do pop(p);
   end;
end.

unit uTElemento; { Almacenada en archivo uTElemento.pas }

interface
   type
     tElemento = record
         titulo: string[255];
         autor:  string[255];
         ISBN:  string[13];
         idioma: string[2];
         num_paginas: integer;
     end;

     tnodo = record
         info: tElemento;
         ant: ^tnodo;
     end;

   procedure assign(var e1: tElemento; e2: TElemento);

implementation

   procedure assign(var e1: tElemento; e2: TElemento);
   begin
     e1:= e2;
   end;

end.

program ejercicio_propuestos_pilas2; { Almacenado en archivo problema3.pas }

uses uPilaGenerica, uTElemento, crt;

procedure recolocar(var p: tPila);
var
   paux: tPila;
   libro_aux: tElemento;
begin
  while not is_empty(p) do begin
       assign(libro_aux,peek(p));
       pop(p);
       push(paux,libro_aux);
  end;
  clear(p);
  p:= paux;
end;

var
   pila_libros: tPila;
   libro: tElemento;

begin
   initialize(pila_libros);
   { repetir dar valores a la variable libro y hacer push varias veces }
   push(pila_libros, libro);
   { ... }
   recolocar(pila_libros);
   writeln('Pila recolocada!!');
   repeat until keypressed
end.
```

EJERCICIO 3

Ampliar la especificación del TAD Pila de enteros añadiendo las siguientes operaciones e implementarlas en ambos enfoques de implementación (con array y con memoria dinámica):

1. `count(pila)`: retorna cuántos elementos tiene la pila

```
SOLUCIÓN: Se proporcionan 3 versiones diferentes, según la implementación interna
(las dos primeras) o independiente de la implementación (la última). Dado que se
piden ambos enfoques, la respuesta correcta al ejercicio implicaría bien hacer las
dos primeras o bien hacer la tercera.

function count(pila: tPilaEnteros): integer;
{ Implementación accediendo a la estructura de la Pila – SOLO para ARRAYs }
begin
  count:= cima;
end;

function count(pila: tPilaEnteros): integer;
{ Implementación accediendo a la estructura de la Pila – Solo para PUNTEROS }
var
  aux: tpilaenteros;
  counter: integer;
begin
  counter:= 0;
  aux:= pila;
  while aux <> nil do
  begin
    aux:=aux^.ant;
    counter:= counter+1;
  end;
  count:= counter;
end;

function count(pila: tPilaEnteros): integer;
{ Implementación usando solo funciones básicas del TAD Pila – Vale para ambas
implementaciones (arrays y punteros) si el contrato es igual }
var
  aux: tPilaEnteros;
  counter: integer;
begin
  counter:= 0;
  initialize(aux);
  while not is_empty(pila) do { Sacamos el contenido de la pila a una pila auxiliar }
  begin
    push(aux,pop(pila));
    counter:= counter+1;
  end;
  while not is_empty(pila) do push(pila,pop(aux));{ Volvemos a llenar la original }
  count:= counter;
end;
```

2. `bottom(pila)`: consulta el elemento más profundo de la pila

```
function bottom(var pila: tPilaEnteros): integer;
{ PRE: pila no vacía }
{ Implementación válida para las dos posibles implementaciones: arrays y punteros }
var
  aux: tPilaEnteros;
  bottom_element: integer;
begin
  initialize(aux);
  while not is_empty(pila) do { Volcamos la pila original sobre la auxiliar }
```

```
    push(aux,pop(pila));
  bottom_element:= peek(aux); { Recordamos el último elemento }
  while not is_empty(aux) do   { Volvemos a poner las cosas en su sitio }
    push(pila,pop(aux));
  bottom:= bottom_element;
end;

function bottom(var pila: tPilaEnteros): integer;
{ PRE: pila no vacía }
{ Implementación muy simple pero solo válida para la implementación con arrays }
begin
  bottom:= pila.stack[1]
end;
```

3. `mount(pila1,pila2)`: poner todos los elementos de la segunda pila encima de la cima actual de la primera respetando el orden de los elementos

```
procedure mount(var pila, pila2: tPilaEnteros);
{ Implementación válida para ambas implementaciones }
var
  aux: tPilaEnteros;
  x: integer;
begin
  initialize(aux);
  while not is_empty(pila2) do {sacamos los elementos de pila2 a una pila auxiliar..}
    push(aux,pop(pila2));
  while not is_empty(aux) do {...y los metemos encima de los que hay en pila}
  begin
    x:= pop(aux);
    push(pila,x);
    push(pila2,x); { también los metemos en pila2 para dejarla como estaba }
  end;
end;
```

4. `npop(pila,n)`: quita tantos elementos de la pila como indica el parámetro entero

```
procedure npop(var pila: tPilaEnteros; n:integer);
begin
  while (not is_empty(pila)) and (n>0) do
  begin
    pop(pila);
    n:= n-1;
  end;
end;
```

5. `summarize(pila,n)`: reemplaza los primeros m elementos de la pila por su suma. Si hay menos de n elementos reemplaza todos los que haya por su suma.

```
procedure summarize(var pila: tPilaEnteros; n:integer);
var
  suma: integer;
begin
  if (n>0) and (not es_vacia(pila)) then
  begin
         suma:= 0;
         repeat
           suma:=suma+peek(pila);
           pop(pila);
           n:= n-1;
         until is_empty(pila) or (n=0);
         push(pila,suma);
  end;
end;
```

EJERCICIO 4

Utilizando una pila, comprobar si una frase es un palíndromo.

```
program ejercicio_pilas_4;
uses uPilaChar, crt;

procedure al_reves(var p: tPilaChar);
//  Este procedimiento tendría que incluirse en el TAD PilaChar ya que accede a su
//  representación interna y por tanto habría que incluirlo en la unit uPilaChar
var aux: tPilaChar;
begin
  initialize(aux);
  while not is_empty(p) do
    push(aux,pop(p));
  p:= aux;
end;

var
  palabra, inverso: string;
  p: tPilaChar;
  i: integer;

begin
  initialize(p);
  write('Introduce una palabra: ');
  readln(palabra);
  inverso:='';
  i:= 0;
  while i<length(palabra) do
  begin
    i:= i+1;
    push(p,palabra[i]);
  end;
  al_reves(p);
  while not is_empty(p) do
  begin
    inverso:= peek(p) + inverso;
    pop(p);
  end;
  if palabra=inverso then writeln('La palabra ', palabra, 'es un palindromo');
  else writeln('La palabra ', palabra, 'NO es un palindromo');
  repeat until keypressed
end.
```